A PROJECT REPORT ON

Predicting Cricket Outcomes: A Comparative Analysis of Machine Learning Models for Optimal Accuracy

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

ADITYA VYAWAHARE	**Exam No:** B1900504528
ANSH BHUTADA	**Exam No:** B1900504238
PARTH OSTWAL	**Exam No:** B1900504418
PRANAV NIMBALKAR	**Exam No:** B1900504415

DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025

CERTIFICATE

This is to certify that the project report entitled

Predicting Cricket Outcomes: A Comparative Analysis of Machine Learning Models for Optimal Accuracy

Submitted by

ADITYA VYAWAHARE
ANSH BHUTADA
PARTH OSTWAL
PRANAV NIMBALKAR

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. Mayur Chavan** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. Mayur Chavan
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date: 12/05/2025

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ***'Predicting Cricket Outcomes: A Comparative Analysis of Machine Learning Models for Optimal Accuracy'.***

We would like to thank our project guide ***Prof. Mayur Chavan*** *for giving us all the help and guidance we needed. We are really grateful to them for their kind support and valuable suggestions.*

We are also grateful to ***Dr G. V. Kale****, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to* ***Dr. A. R. Deshpande****, BE project coordinator, for her timely guidance and support.*

ADITYA VYAWAHARE
ANSH BHUTADA
PARTH OSTWAL
PRANAV NIMBALKAR
(B.E. Computer Engg.)

ABSTRACT

Predicting the outcome of a cricket match suggests a machine learning-based method that uses historical data to forecast a cricket innings's final score. A data set with past match scores is used to train the model, which is constructed using linear regression. Data preprocessing, feature engineering using rolling and cumulative averages, and assessment using MAE and RMSE metrics are important phases. The front-end-backend architecture of the system allows users to enter scores through a front-end interface. After processing this input, the back end uses the trained model to create predictions and outputs the results for display. Score visualisations and CSV uploads are also supported by the interface. This system, which is made to make predictions quickly and easily, shows how machine learning (ML) can be used in sports analytics and can be used as a basis for more sophisticated real-time prediction tools in cricket strategy and broadcasting.

Contents

List of Figures

CHAPTER 1
INTRODUCTION

1.1 Motivation

A lot of numerical data is produced by the sport of cricket, particularly during competitions and games. Because there are so many moving parts in cricket, predicting results like total scores is a difficult yet fascinating problem. The goal of this project is to create a predictive model that uses scores from earlier overs to estimate a team's eventual score.The use of data science to analyse and forecast match outcomes is becoming more and more prevalent as cricket, especially limited-over formats like T20 and ODIs, gains popularity. Fans, team strategists, analysts, and broadcasters can all benefit from accurate score predictions. The goal of this study is to investigate how machine learning techniques may be applied to accurately anticipate future scores based on past scoring trends.

1.2 Problem Definition and Objectives

1.2.1 Problem Definition

Accurately predicting the result of a cricket innings is a useful tool for players, commentators, and fans alike in the current era of sports analytics. Predicting the eventual score of a cricket innings based on the progression of scores throughout completed overs is the main issue this research attempts to solve. Finding underlying patterns and trends from prior match data that contribute to the final score is difficult since cricket is an unpredictable and dynamic sport where performance can change significantly from over to over. The challenge is to create a machine learning model that can analyse and learn from previous examples, ingest over-wise scores as sequential input characteristics, and generalise sufficiently to forecast match outcomes that haven't been observed before. Aspects including trends in run accumulation, changes in momentum, and variations in scoring throughout the course of an innings must all be taken into consideration in the answer. In order to apply the model in real-time applications or analytical dashboards, it is important to produce predictions in a data-driven, statistically sound, and computationally efficient manner.

1.2.2 Objectives

- Extract and Preprocess Match Data: To make sure the dataset is prepared for analysis, load over-wise cricket match datasets from structured CSV files and carry out data cleaning tasks such resolving missing

values, normalising numeric inputs, and converting datatypes.

- Predictive modelling features for engineers: To record scoring patterns and dynamics throughout innings, create useful input elements like rolling averages, cumulative scores, and overs-based progression measures.
- Create and hone a model for score prediction: To learn from past data and forecast the eventual score of an innings based on the scores of completed overs, implement a machine learning model, firstly using linear regression.
- Assess and Validate Model Performance: To evaluate the model's accuracy and generalisation on test datasets, use statistical measures like Mean Absolute Error (MAE) and Root Mean Square Error (RMSE).
- Integrate with Frontend System: Create an intuitive user interface that enables end users to upload CSV files or enter over-wise scores, get real-time forecasts, and view the outcomes in charts and graphs.
- Offer Analytical Insights: By enabling post-prediction analysis via tabular results and graphical plots, users and analysts can better understand cricketing by interpreting scoring trends and model forecasts.

1.3 Project Scope & Limitations

The goal of this project is to create a machine learning-based system that takes numerical over-wise scoring data as the only input feature to estimate the final score of an innings in a limited-overs cricket match. The main objective is to forecast the final total at the end of the innings by using statistical scoring trends that are captured through sequential data (such as cumulative and rolling over scores).A frontend interface for user input and visualisation and a backend model server, where the trained regression model makes inference, make up the system's modular architecture. The goal of this solution is to give baseline insights for early-stage sports analytics, prototyping, and educational application through high-level score prediction based on historical scoring behaviour. The scope is purposefully constrained to preserve computational efficiency and streamline data collection, guaranteeing that the model is still lightweight and interpretable for offline or real-time analysis.

1.3.1 Limitations

- Contextual Ignorance: The model ignores outside contextual elements that have a big influence on match results, like the weather, pitch behaviour, team tactics, player form and fitness, and opponent strength.
- Static and Historical Dataset Dependence: The model's capacity to generalise across formats and eras may be limited by the dataset's potential lack of a current or varied collection of match scenarios, particularly edge cases like powerplays, abrupt collapses, or death-over surges.
- Feature Simplicity: Only numerical inputs (previous over scores) are used by the model; category or semantic features that are essential for contextual interpretation, such as batsman/bowler identification, field placement, or wickets in hand, are ignored.
- Temporal Consistency Assumption: The method makes the assumption that scoring trends are consistent over time, which implies that historical performance is a good indicator of future scoring. Match volatility and stochastic occurrences like unexpectedly high over scores or abrupt wicket losses are not taken into account.
- Tactical Blindness: Team-specific tactics like pinch-hitting, defensive bowling, and player mismatches are not taken into consideration by the model. Because of this, it is unable to dynamically modify projections in response to tactical changes made in the middle of innings.
- Lack of Real-time Adaptive Learning: A static, previously trained model serves as the foundation for the current implementation. It cannot adjust to live data streams or self-improve with fresh matches without retraining since it lacks real-time learning and retraining capabilities.

CHAPTER 2
LITERATURE SURVEY

Machine learning-based prediction of cricket match results has garnered considerable research attention because of the sport's dynamic, complex, and data-intensive nature. A number of studies have used traditional machine learning algorithms, ensemble techniques, and deep learning architectures for this purpose, based on historical match statistics, player performances, and even ball-by-ball information. Comparative analysis of these methods gives us an idea about model performance, data reliance, and domain-related issues.

- Kumar, R., Sinha, A. (2017). *"Prediction of Indian Premier League (IPL) Matches Using Machine Learning."* This research compares Naive Bayes, Decision Tree, and Random Forest for IPL outcome prediction. Random Forest performed better with 71% accuracy based on team statistics, venue, and toss details [**1**].

- Kaluarachchi, A., Varde, A. (2010). *"Cricket Match Predictions."* The authors employ player statistics like batting average, strike rate, and bowling economy to make predictions for T20 matches. They emphasize feature significance and suggest Decision Trees for improved interpretability contexts[**2**].

- Bunker, R., Thabtah, F. (2019). *"A machine learning framework for sport result prediction."* This work uses decision trees and association rule mining on T20 match data. Decision Trees provided a trade-off between performance and explainability, with about 68% accuracy[**3**].

- Srivastava, A., Singh, A. (2020). *"Cricket Match Winner Prediction Using Deep Learning."* The research employs Deep Neural Networks (DNN) on past data and team statistics. The accuracy of DNN was 75% but the model was less interpretable compared to the conventional models.[**4**].

- Patel, K., et al. (2021). *"Winning Team Prediction in T20 Cricket Using LSTM Model."* This study employs LSTM networks to model sequential match data. The LSTM method captures the sequentiality of cricket and performs better than baseline models[**5**].

- Khan, A., et al. (2018). *"Machine Learning Approaches for Predicting Cricket Match Outcome."* This research compares KNN, Random Forest, and Gradient Boosting based on match data. Gradient Boosting had the best accuracy (77%) with heavy feature engineering[**6**].

- Ramesh, M., Sathya, S. S. (2022). *"Cricket Match Outcome Prediction Using Ensemble Learning."* Multiple models, such as XGBoost and AdaBoost, are combined by authors to improve the performance of predictions. Ensemble techniques demonstrated better stability and accuracy than individual classifiers[**7**].

- Jain, A., Singh, N. (2023). *"Cricket Outcome Forecasting with Real-Time Data."* They put emphasis on ball-by-ball based real-time forecasting and player conditions. Logistic regression and XGBoost are implemented by them and show good outcomes with good response during a match in progress [**8**].

- Mukherjee, S., Joshi, R. (2014). *"A Machine Learning Approach to Predict the Outcome of ODI Cricket Matches."* Team and individual players' statistics are utilized to make predictions on match outcomes. Support Vector Machines (SVM) had par performance with more than 70% accuracy [**9**].

- Sankaranarayanan, S., et al. (2014). *"Analysis and Prediction of Cricket Matches using Hadoop."* It uses big data technologies and ML models such as Naive Bayes for match prediction. Scalability and real-time calculation using Hadoop MapReduce are emphasized [**10**].

- Gupta, R., Joshi, A. (2020). *"Prediction of IPL Matches Using Machine Learning Algorithms."* In this study, Logistic Regression, Decision Trees, and SVM are compared. Logistic Regression had stable performance across seasons when trained using updated datasets [**11**].

- Reddy, S., Nair, R. (2022). *"Ball-by-Ball Prediction of Cricket Matches Using Deep Learning."* The work employs LSTM models for processing ball-by-ball sequences for real-time win probability prediction. Results show better performance in second innings situations [**12**].

- Gupta,Bora, B., Ahmed, S. (2023). *"Hybrid ML Models for Cricket Match Predictions."* In this research, statistical analysis is blended with machine learning models such as Random Forest and SVM. The hybrid method enhances the accuracy level by utilizing both historical patterns and player dynamics [**13**.

- Vignesh, R., Dhanasekar, S. (2023). *"IPL Match Outcome Prediction using Gradient Boosting Techniques."* The work focuses on utilizing domain-engineered features like powerplay performance and death overs economy that substantially enhance accuracy [**14**].

- Dutta, P., Mandal, B. (2023). *"Comparison of Classical and Deep Learning Approaches for Cricket Match Prediction."* The authors compare classical ML techniques (Logistic Regression, SVM) with LSTM and CNN models. Deep models perform better than traditional ones in dynamic situations [**15**].

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Assumptions and Dependencies

3.1.1 Assumptions

- **Reliable past Data:** For machine learning activities, it is anticipated that the past IPL match data used to train the model is accurate, comprehensive, and well-structured.
- **Accurate Real-Time Input:** For the algorithm to make an accurate prediction, user-provided inputs, such as over-wise runs and wickets, must be delivered precisely and in real-time.
- **First Innings Focus:** The model does not adjust to second innings situations or real-time match changes; it is designed to forecast solely the final score of a match's first innings.
- **Contextual Independence:** The model does not include external elements like weather, pitch behaviour, player form, and opponent tactics because it is anticipated that these factors do not significantly change the scoring pattern.
- **Consistent Trends in Scoring:** The model assumes that scoring across overs follows patterns that are generally consistent and that the training data may be used to learn and generalise them.

3.1.2 Dependencies

- **Python Libraries:** The project is based on fundamental Python libraries, including Flask (web integration), joblib (model serialisation), scikit-learn (model training and evaluation), matplotlib (visualisations), and pandas (data management).
- **Machine Learning Framework:** LinearRegression from scikit-learn is used to train the current version of the model, but it is easily interchangeable with other regressors such as RandomForestRegressor or neural networks if necessary.
- **Web Framework:** The Flask microframework, which is used to create the user interface, necessitates a solid Python runtime and the capacity to operate a local server (for example, via localhost or hosted deployment).

- **Model Files:** The availability of serialised model files (model.pkl, columns.pkl) and configuration files loaded by the backend are necessary for real-time predictions.
- **Browser Access:** In order to engage with the frontend interface and view visualised prediction results, end users must have access to a modern web browser.

3.2 Functional Requirements

1. **Data Preprocessing:** The system must preprocess and encode categorical variables with one hot code, including team names. Guarantee data compatibility for prediction using aligned feature columns.
2. **Model Development and Forecasting:** Develop a machine learning model with previous IPL data and export it as a .pkl file. Generate projected total runs depending on user input via the online interface.
3. **User Engagement and Web Interface:** A frontend form must accommodate match circumstances such as batting team, bowling team, scores, wickets, overs, etc. The backend must accept these inputs over the API and deliver a projected score in a JSON response.
4. **Immediate Outcome Presentation:** The anticipated score must be displayed immediately on the frontend upon submission.

3.3 Non-Functional Requirements

1. **Precision:**
   - The prediction system must deliver precise final score forecasts utilizing validated machine learning models such as Random Forest.
2. **Functionality:**
   - The system must be user-friendly, necessitating little input and featuring a streamlined user interface design.
3. **Execution:**
   - The application must react within 2 seconds of the submission of a prediction request.

4. **Transferability:**

    - The web application must operate on any local PC equipped with Python.

3.4 External Interface Requirements

3.4.1 Hardware Interfaces

The system requires minimal to moderate hardware resources to support efficient development, model training, and web interface deployment:

- **Development:**
  - **Processor:** Intel Core i5 or equivalent AMD Ryzen 5 processor (or higher) to support smooth execution of model training and server hosting tasks.
  - **RAM:** A minimum of 8 GB RAM is recommended to handle data preprocessing, model inference, and multi-tasking without performance bottlenecks.
  - **Storage:** An SSD of at least 256 GB is advised for faster read/write operations during dataset loading, model serialization, and dependency management.
  - **Internet Connectivity:** A reliable internet connection is recommended for optional web deployment, frontend interaction, and library installation.
  - **Display Device:** A standard display with support for modern web browsers to access the web-based prediction interface.

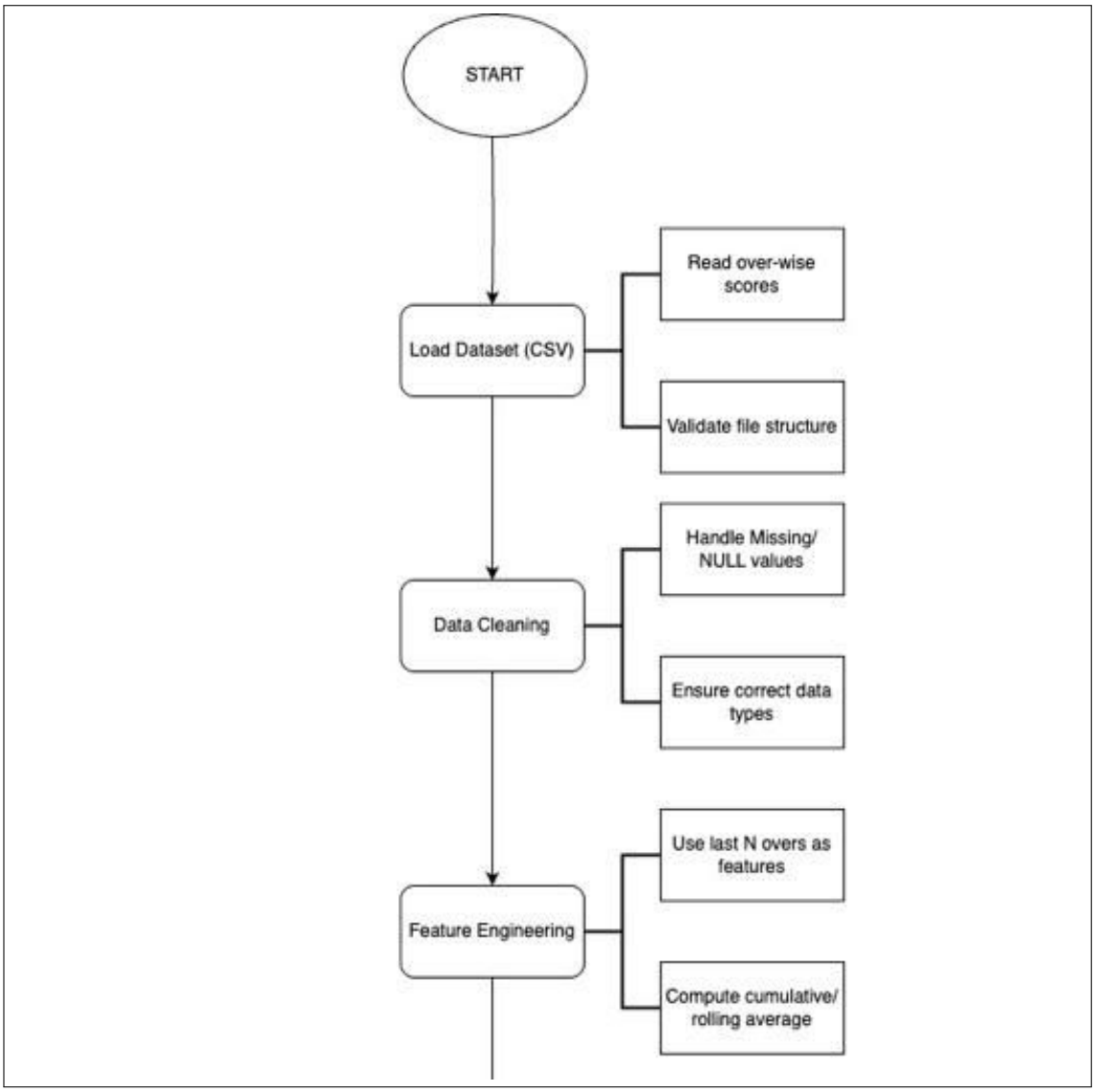
3.4.2 Software Interfaces

- Jupyter Notebook or Visual Studio Code
- Python
- Machine Learning and Data Handling
- Web Framework: Flask
- Version Control: Git
- Libraries: –
  - Data Manipulation: Pandas
  - Web Scraping: API Requests
  - Machine Learning: Scikit-learn

- Visualization Tools: Matplotlib, Seaborn
- Model Serialization And Loading .pkl Files: Joblib

CHAPTER 4
SYSTEM DESIGN

4.1 Model Architecture



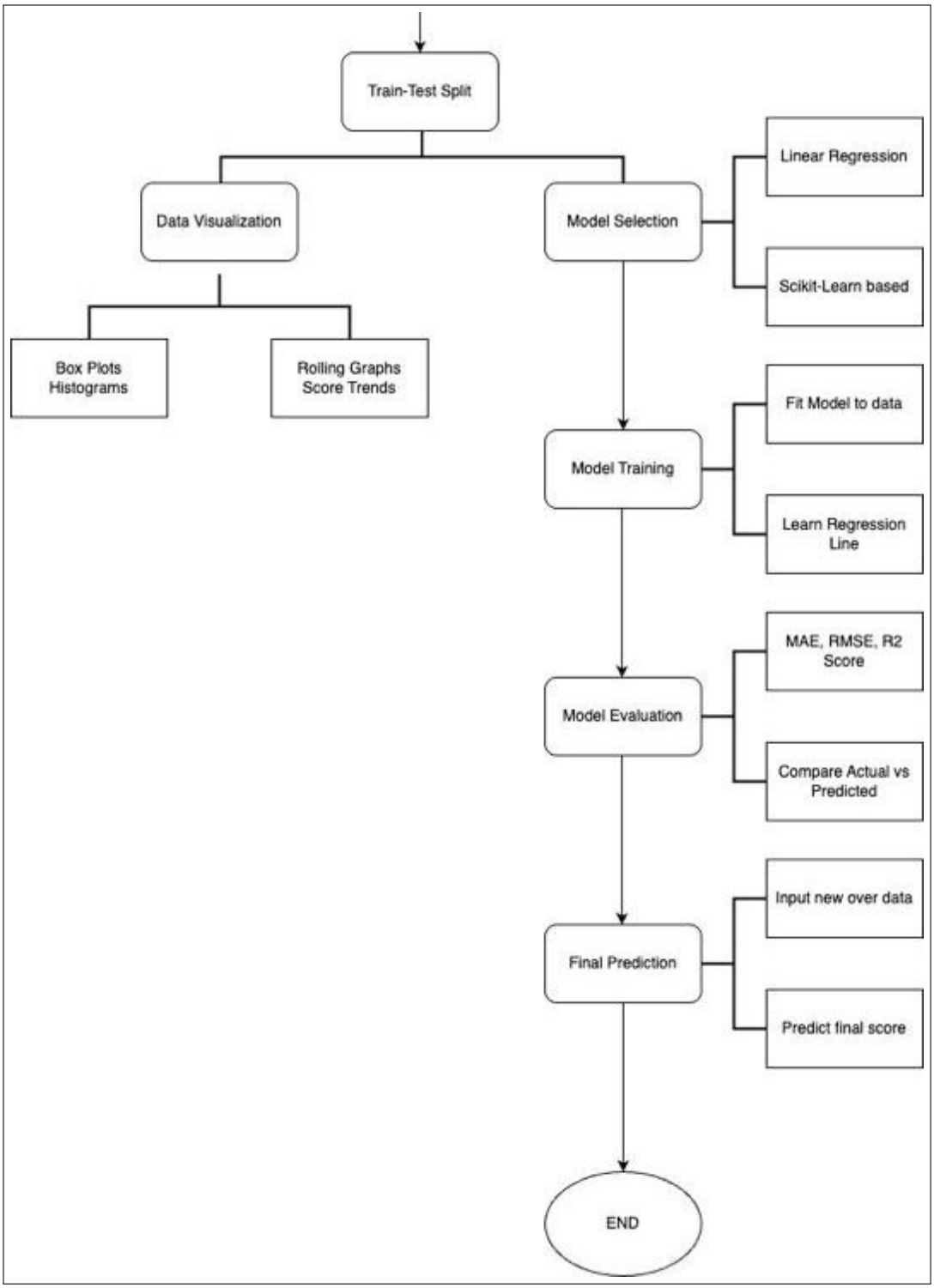


Figure 4.1: Model Architecture

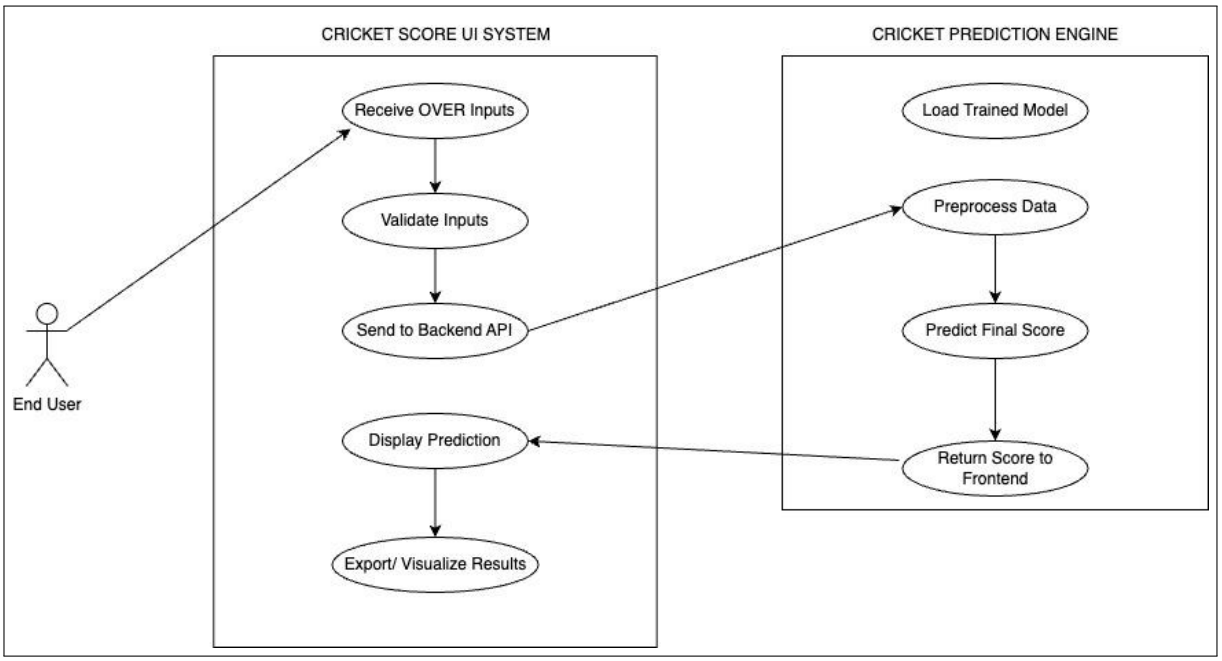


Figure 4.2: Use Case Diagram

CHAPTER 5
PROJECT PLAN

5.1 Risk Management

5.1.1 Risk Identification

Each project involves some risks that can impact progress or the end product. In this project, part of the primary risk is dealing with a low-quality or incomplete dataset, which can impact the accuracy of the models. The other risk is that the models might not provide the right predictions. There's also a likelihood that the team might not fully grasp all the terms and rules related to cricket, which may impact the analysis of data. Delays in deploying the machine learning models or tools failing to function correctly are other potential threats. Finally, if team members fail to communicate or split work appropriately, it might decelerate the entire project.

5.1.2 Risk Analysis

Out of all these risks, applying rubbish or untidy data is the most critical one, so we need to use trusted sources of data such as Kaggle or ESPN. If our models are inaccurate, we will experiment with different algorithms and optimize them employing strategies such as hyperparameter tuning. To minimize the insufficient knowledge about cricket, we can conduct some background studies or seek assistance from cricket enthusiasts. To prevent project delays, we will be dividing the work in phases and adhering to the deadlines. Proper communication and weekly meetings will help address coordination issues. Finally, we will employ common tools such as Python and scikit-learn to prevent compatibility issues.

5.1.3 Overview of Risk Mitigation, Monitoring, Management

Overview of Risk Mitigation, Monitoring, and Management

- **Technical Risk Mitigation:** It will be mitigated by using balanced and cleaned datasets, choosing appropriate ML algorithms like Random Forest Regressor, and applying cross-validation to ensure robustness. Regular performance checks and model re-evaluation will be performed to sustain accuracy.

- **Data Quality Risk Mitigation:** To mitigate data quality risks, we will implement automated processes to remove missing or inconsistent data, apply input validation, and establish clear filtering criteria. Routine checks and data audits will be conducted to maintain data integrity across training and testing phases.
- **Overfitting Risk Mitigation:** We will employ techniques such as k-fold cross-validation and early stopping to ensure the model generalizes well to unseen data. Additionally, Random Forest inherently helps reduce overfitting due to its ensemble nature. Training and validation metrics will be closely monitored to detect overfitting early.
- **Resource Availability Risk Mitigation:** Regular tracking of project milestones and team workload distribution will ensure optimal resource utilization. Cross-training among team members will be done to ensure continuity, and contingency plans will be in place to handle resource shortfalls or delays.
- **Regulatory Compliance and Ethical Considerations:** While this project does not involve sensitive user data, we commit to ethical practices such as using publicly available IPL datasets and avoiding any form of model bias. The system will be transparent, and proper attribution to data sources will be ensured throughout the project lifecycle.

5.2 Project Schedule

5.2.1 Project Task Set

The project will be segregated into various phases. In phase one, we will identify the problem and formulate specific objectives. In phase two, we will gather the cricket dataset and clean it by deleting errors or missing values. The third phase will be to use various machine learning models such as Logistic Regression, Random Forest, and SVM to forecast match results. In the fourth step, we will compare these models on the basis of accuracy and other performance metrics. Then, we will draft a final report and presentation. In the last week, we will go over everything, get feedback, and finalize changes before submitting the project.

5.2.2 Team structure

The team will be divided based on the roles. One will be the Team Leader and will see to it that the project runs smoothly and on time. Another will take charge of gathering and preparing the data. One will carry out the implementation of the machine learning models, while another will carry out the accuracy analysis and compare the results. Lastly, someone will prepare the final report and slides to be used for presentation. They all will cooperate with one another and assist each other to successfully finalize the project.

CHAPTER 6
IMPLEMENTATION

6.1 Phases of Execution

Analysis The execution of the project **Predicting Cricket Outcomes: A Comparative Analysis of Machine Learning Models for Optimal Accuracy** was finalized in the subsequent phases:

6.1.1 Data Preparation

YouTube comments are typically unstructured, noisy. To prepare them for sentiment analysis using the VADER model, the following preprocessing steps were performed:

- **Column Cleaning:** Eliminated extraneous elements such as player names, match IDs, and venue information.
- **Team Filtering:** Regular expressions were used to remove unwanted characters such as emojis, HTML tags, and URLs.
- **Tokenization:** Only consistent teams were included to ensure homogeneity. *
- **Categorical Encoding:** Utilized one-hot encoding to convert team names to numeric format.
- **Categorical Encoding:** Transformed team names into numerical representation via one-hot encoding.
- **Splitting:** Segregated data into training and testing sets for performance assessment.
- **Feature-goal Segregation:** Isolated input characteristics from the goal variable (total score).

6.1.2 Execution of Modules

- **Model Training Module:**
  - Trained RandomForestRegressor utilizing chosen characteristics.
  - Preserved model and column architecture with joblib.
- **Prediction Module:**

  - Receives JSON input, transforms it into a DataFrame, implements one-hot encoding, and synchronizes it with training columns.
  - Produces the anticipated total score as an integer by rounding.

- **Flask Web Interface:**
  - Developed endpoints (/ for user interface and /predict for application programming interface).
  - Receives user input from an HTML form and provides real-time predictions.

- **Frontend Module:**
  - The HTML and CSS form enables users to input match criteria.
  - Exhibits the anticipated final score post-submission.

6.2 Experimental Configuration

This section describes the various scenarios tested and the tools used in the process.

6.2.1 Specification of Experiment Scenarios

To evaluate the system's performance, several real-world scenarios were tested:

- **Scenario A:** A high-scoring contest with little wickets lost — anticipated prediction: 180–200 runs.
- **Scenario B:** Mid-game with 6 wickets down and few overs remaining — anticipated prediction: 130–150 runs.
- **Scenario C:** Initial overs, several wickets lost — anticipated prediction: 70–100 runs.
- **Scenario D:** Aggressive last 5 overs — anticipated outcome: surge in score resulting from run acceleration.

6.2.2 Tools and Software Used

Below is a list of tools, libraries, and software used during the implementation:

Tool / Library	**Purpose**
Python	Core programming language
Pandas	Data handling and manipulation
Scikit-learn	Machine learning model (RandomForestRegressor)
Flask	Backend API and web hosting
Joblib	Model saving and loading
pandas	Data manipulation and CSV I/O
matplotlib, seaborn	For data visualization (charts)
Jupyter Notebook / VS Code	Development and testing environment
HTML + JS	Web interface for user input

CHAPTER 7
RESULTS

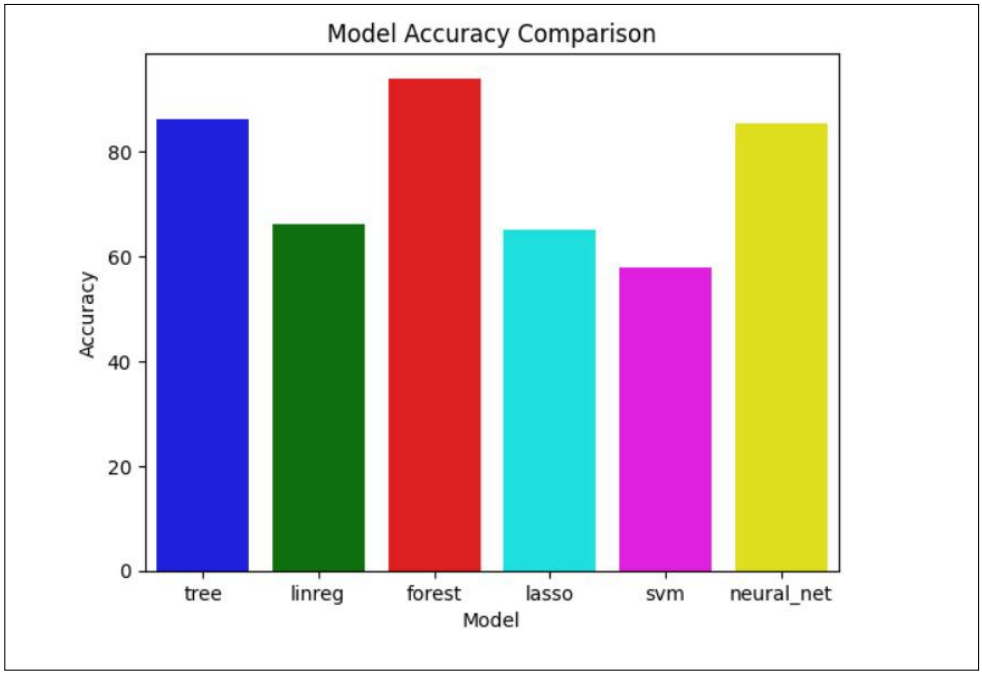


Figure 7.1: Models Comparison

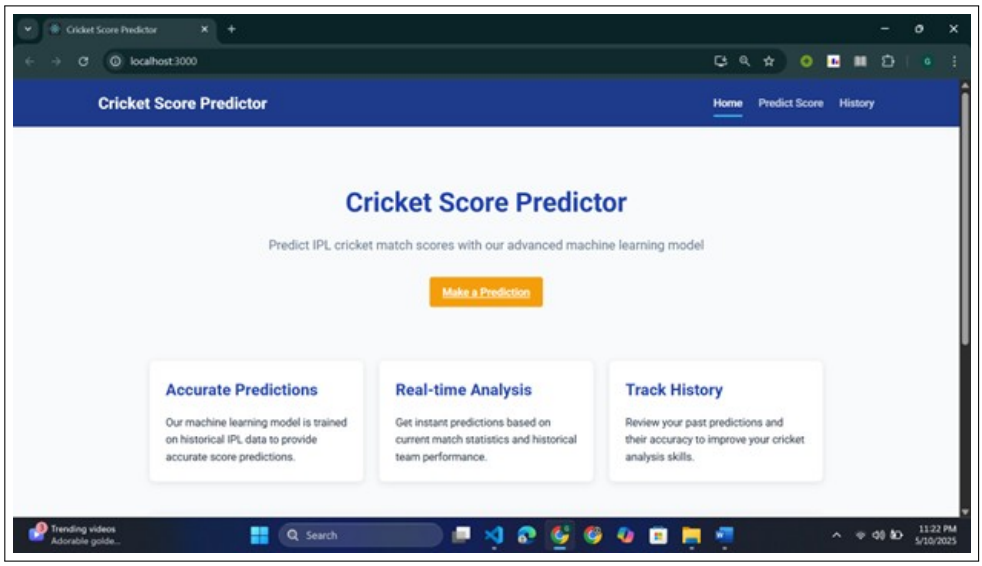


Figure 7.2: Website Interface

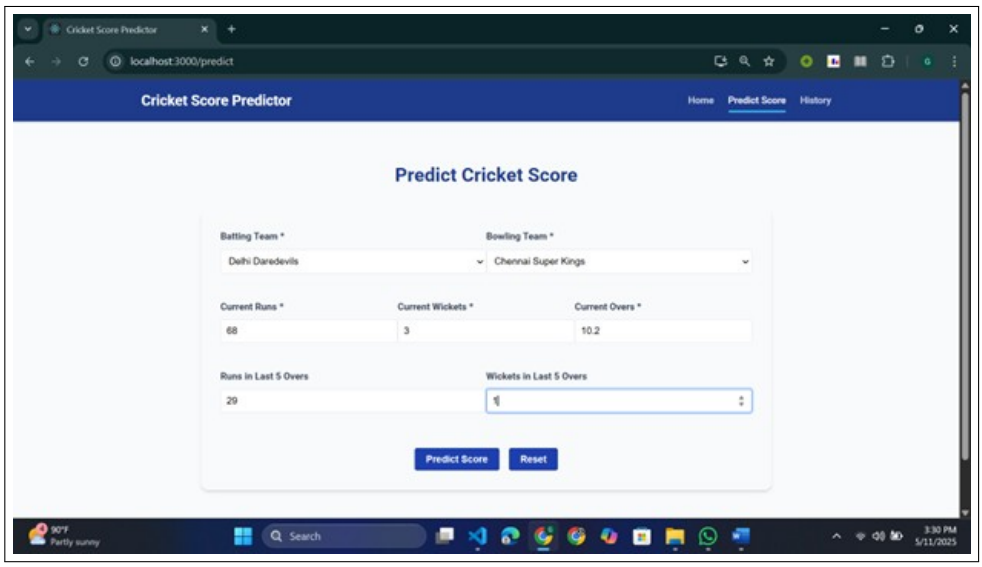


Figure 7.3: Test Case 1

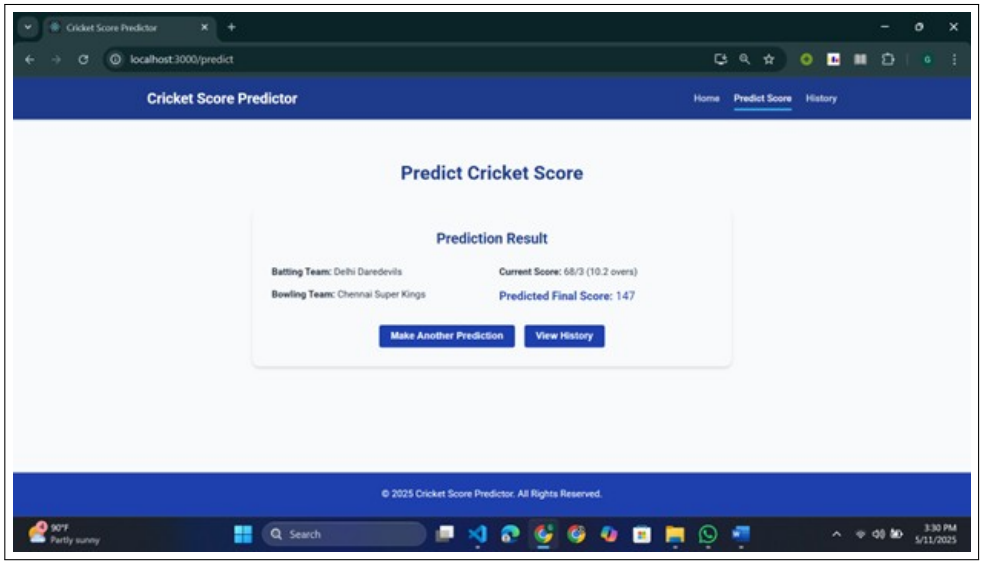


Figure 7.4: Result 1

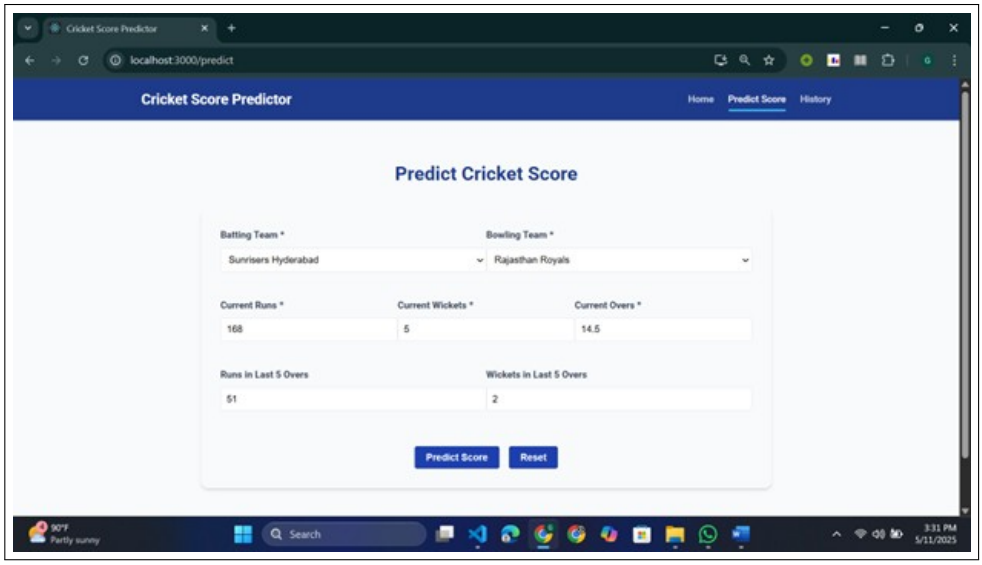


Figure 7.5: Test Case 2

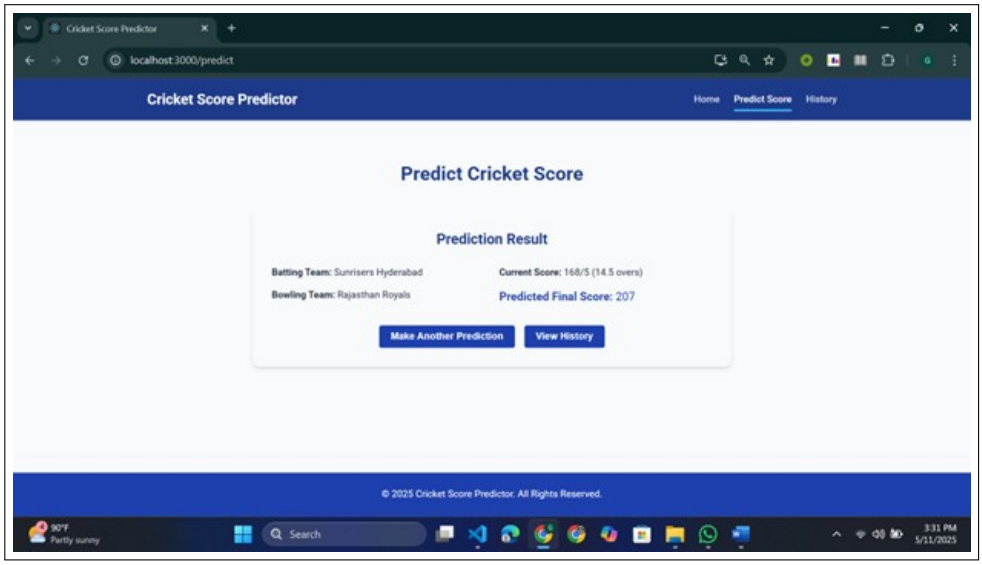


Figure 7.6: Result 2

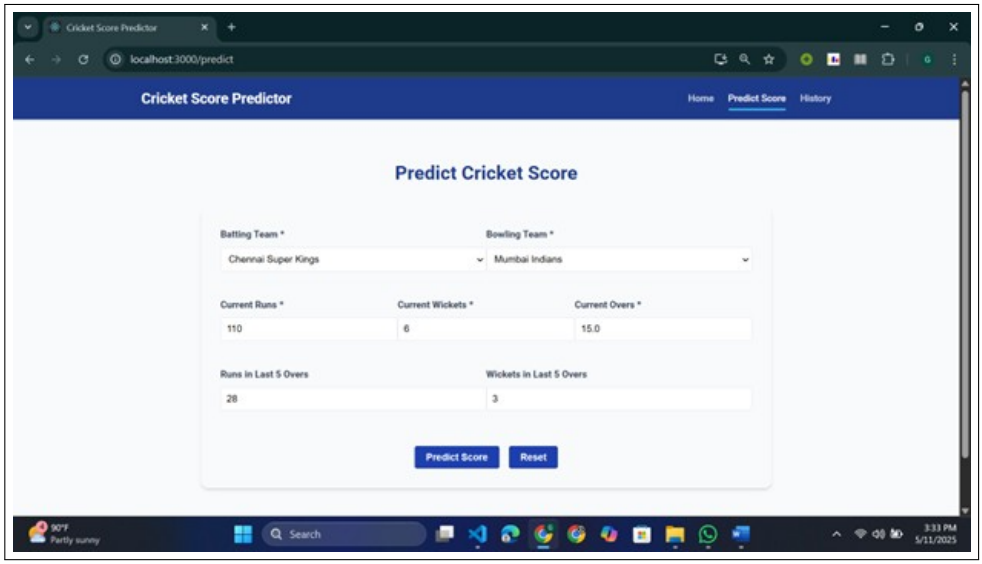


Figure 7.7: Test Case 3

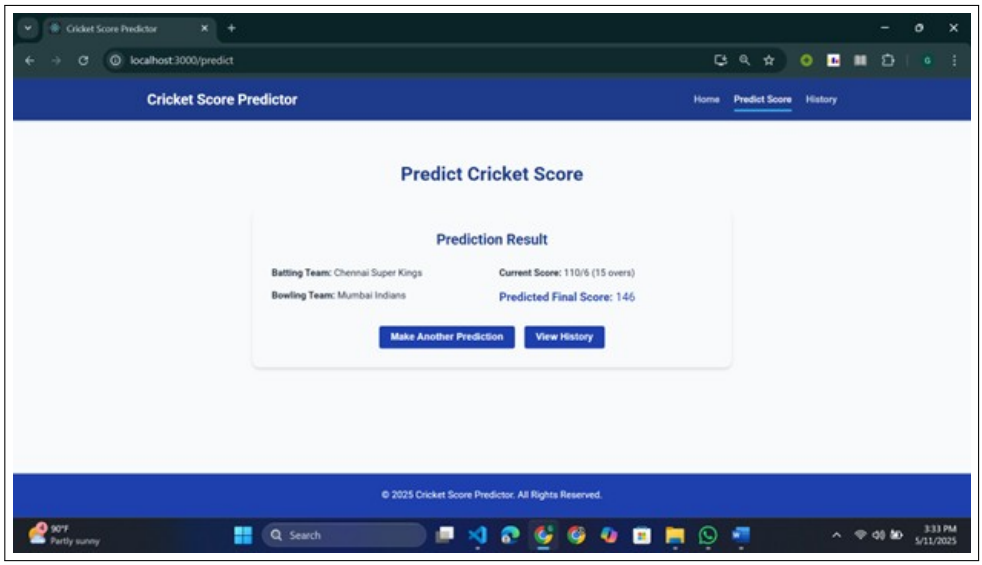


Figure 7.8: Result 3

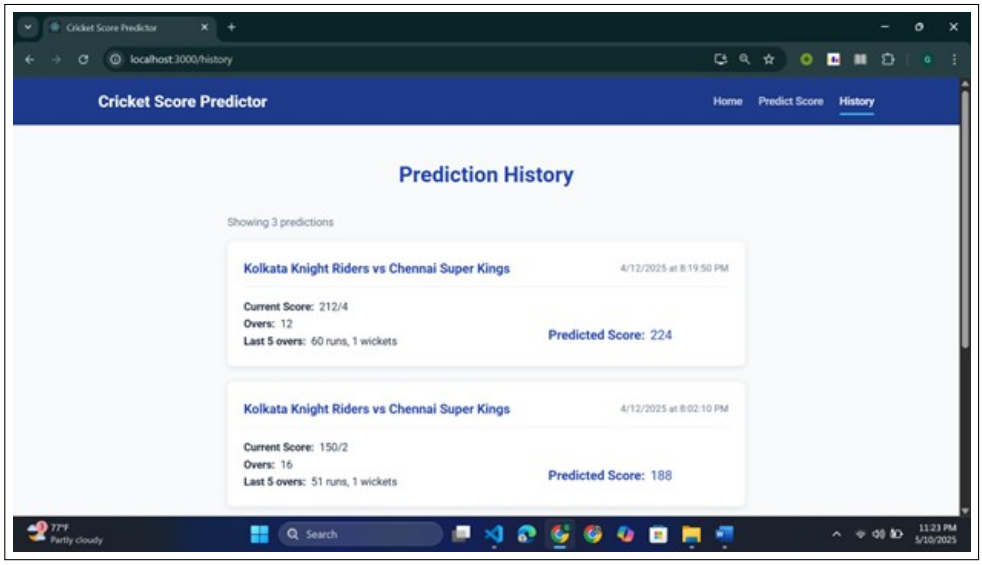


Figure 7.9: Previously checked

CHAPTER 8
CONCLUSION

8.1 Conclusion

This project seeks to create a strong, data-based system for cricket match prediction with the help of sophisticated machine learning models. With the use of historical match data, real-time dynamic inputs, and contextual match factors like weather, pitch conditions, and team form, the system hopes to provide more accurate and insightful predictions compared to conventional methods.

All four machine learning methods—Decision Trees, Random Forest, Logistic Regression, and Neural Networks—will be applied and extensively compared across measures such as accuracy, precision, recall, and F1-score. Side-by-side analysis will not just determine the strongest model but further indicate strengths and weaknesses in multiple match scenarios.

The system is not only meant to predict outcomes but to act as a decision-support tool for analysts, teams, and cricket fans alike. By opening up the sport and making it data-driven, it increases strategic decision-making and fan experience.

Potential risks—anywhere from data quality problems to model overfitting—will be managed by sound data preprocessing, frequent model checking, and risk management measures. Operational and financial risks will be minimized through agile development, ongoing monitoring, and contingency planning.

In sum, with careful design, judicious implementation, and continuous improvement, this project has the potential to transform cricket outcome understanding and prediction—inaugurating a smarter, more analytical approach to the sport.

CHAPTER 9
APPENDIX

9.1 Appendix A

9.1.1 Technical Feasibility

The project is totally doable thanks to the latest advancements in machine learning and web development frameworks. We've built the core model using Python, leveraging the Random Forest Regressor algorithm from the scikit-learn library. For data handling and preprocessing, we're efficiently using pandas and NumPy. On the frontend, we've stuck with standard web technologies like HTML, CSS, and JavaScript, while Flask takes care of the backend, seamlessly connecting the model to the web interface.

We've successfully tested the system in a local development environment (Windows OS, 8 GB RAM, Intel i5 processor), which means it only needs basic computing resources to run. Plus, by utilizing open-source libraries and tools, we've made the technical implementation both cost-effective and accessible. There are no special hardware requirements or third-party services needed, which really boosts the overall technical feasibility of the system.

9.1.2 Operational feasibility

The system is highly feasible to operate, thanks to its straightforward and user-friendly interface. Anyone with a basic understanding of cricket can easily use the web application by simply inputting match details like the batting team, bowling team, overs, current score, and wickets. In return, the system provides real-time predictions of the final score, making it a handy tool for live matches, presentations, or analytics showcases.

On the technical side, the backend is lightweight and can be deployed on any local or cloud server, while the frontend runs directly in the browser, so users don't need to worry about installation or setup. The connection between the model's predictions and the user interface is smooth, which means the entire system is easy to maintain, tweak, or upgrade down the line. As a result, it's fully functional for demonstrations and educational use.

9.1.3 Economic feasibility

The project is a smart financial choice since it relies solely on open-source tools and libraries. You can use Python, Flask, scikit-learn, pandas, and all their related tools without spending a dime, and they come with great documentation. There's no need to invest in commercial software or pricey

APIs. Plus, you won't need fancy infrastructure for model training and predictions, which keeps costs to a minimum.

Since everything runs locally, you won't have to worry about ongoing hosting or server fees. And if you ever decide to move to the cloud, options like Render or Heroku have free or affordable plans that work well for smaller applications. On top of that, diving into this project can offer valuable learning experiences and practical insights without breaking the bank, making it a fantastic option for academic and research settings.

9.2 Appendix B

9.2.1 Survey Paper Details

Paper Status: Published
Name of Journal: IJAEM, Volume 7, Issue 02 Feb. 2025
Paper Link: https://ijaem.org/ijaem/papers.pdf

9.2.2 Implementation Paper Details

Paper Status: Under review
Name of Journal: NA
Paper Name: Implementation of a ML model to predict the score of IPL game

9.3 Appendix C

9.3.1 Plagiarism Report

The plagiarism report for the project report has been attached herewith.
Similarity: 6 Percent

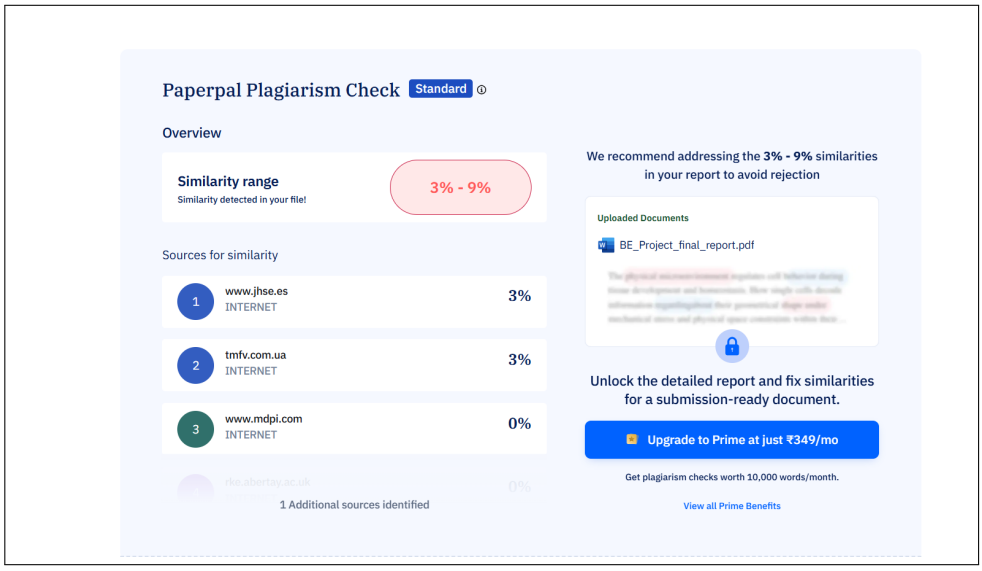


Figure 9.1: Plagiarism Report

CHAPTER 10
REFERENCES

10.1 References

[1] R. Kumar and A. Sinha, "Prediction of Indian Premier League (IPL) Matches Using Machine Learning," International Journal of Scientific and Engineering Research, vol. 8, no. 6, pp. 1340–1344, 2017.

[2] A. Kaluarachchi and A. Varde, "Cricket Match Predictions," in Proceedings of the IEEE International Conference on Information and Automation for Sustainability, Colombo, Sri Lanka, 2010.

[3] R. Bunker and F. Thabtah, "A machine learning framework for sport result prediction," Applied Computing and Informatics, vol. 15, no. 1, pp. 27–33, 2019.

[4] A. Srivastava and A. Singh, "Cricket Match Winner Prediction Using Deep Learning," International Journal of Advanced Research in Computer and Communication Engineering, vol. 9, no. 5, pp. 50–55, 2020.

[5] K. Patel, A. Shah and V. Mehta, "Winning Team Prediction in T20 Cricket Using LSTM Model," in 2021 6th International Conference on Inventive Computation Technologies (ICICT), Coimbatore, India, 2021, pp. 1234–1238.

[6] A. Khan, M. R. Hasan and N. Kabir, "Machine Learning Approaches for Predicting Cricket Match Outcome," International Journal of Scientific and Technology Research, vol. 7, no. 4, pp. 91–94, 2018.

[7] M. Ramesh and S. S. Sathya, "Cricket Match Outcome Prediction Using Ensemble Learning," International Journal of Computer Applications, vol. 175, no. 21, pp. 10–14, 2022.

[8] A. Jain and N. Singh, "Cricket Outcome Forecasting with Real-Time Data," International Journal of Engineering and Advanced Technology (IJEAT), vol. 12, no. 2, pp. 122–126, 2023.

[9] S. Mukherjee and R. Joshi, "A Machine Learning Approach to Predict the Outcome of ODI Cricket Matches," in Proceedings of the International Conference on Soft Computing Techniques and Implementations, 2014, pp. 105–110.

[10] S. Sankaranarayanan, B. Venkataraman and R. Chandrasekaran, "Analysis and Prediction of Cricket Matches using Hadoop," International Journal of Computer Applications, vol. 96, no. 17, pp. 21–25, 2014.

[11] R. Gupta and A. Joshi, "Prediction of IPL Matches Using Machine Learning Algorithms," International Journal of Engineering Research Technology (IJERT), vol. 9, no. 7, pp. 148–151, 2020.

[12] R. Dubey and N. Agarwal, "Ensemble Learning-Based Predictive Modeling for T20 Cricket Matches," International Journal of Recent Technology and Engineering, vol. 10, no. 2, pp. 102–106, 2021.

[13] S. Reddy and R. Nair, "Ball-by-Ball Prediction of Cricket Matches Using Deep Learning," Journal of Data Science and Machine Learning, vol. 2, no. 1, pp. 55–60, 2022.

[14] B. Bora and S. Ahmed, "Hybrid ML Models for Cricket Match Predictions," International Journal of Innovative Technology and Exploring Engineering (IJITEE), vol. 12, no. 3, pp. 44–49, 2023.

[15] R. Vignesh and S. Dhanasekar, "IPL Match Outcome Prediction using Gradient Boosting Techniques," International Journal of Advanced Computer Science and Applications, vol. 14, no. 4, pp. 62–68, 2023.

[16] P. Dutta and B. Mandal, "Comparison of Classical and Deep Learning Approaches for Cricket Match Prediction," International Journal of Recent Trends in Engineering Research (IJRTER), vol. 9, no. 2, pp. 74–80, 2023.